

# Week 4 - Lecture 7 - Problem: Processing Student Scores

---

## ILO

- **KU2:** Choose appropriate data types and data structures for different kinds of data.
- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, generators, and exceptions.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS9:** Use standard libraries and follow good programming practices.

## Objectives

After completing this problem, you should be able to:

1. Process a matrix using nested loops.
2. Write a generator that yields values one at a time.
3. Use `try` and `except` to handle invalid data.
4. Calculate a result from values produced by a generator.
5. Explain the difference between control flow and data flow in the program.

## Background

A teacher stores the scores from several assignments in a matrix. Each row represents one student, and each value represents one assignment score. The scores are strings because they may have been read from a text file.

Some entries may be invalid, such as `"missing"` or `"absent"`. Invalid entries should be skipped instead of stopping the program.

Example matrix:

```
scores = [  
    ["12", "15", "missing"],  
    ["8", "10", "9"],  
    ["18", "absent", "13"],  
]
```

## Task

Write a Python program that processes the student-score matrix.

### Part 1: Write a score generator

Write a generator function `valid_scores(matrix)`.

The generator should:

1. Visit every value in every row.
2. Try to convert the value to an integer.
3. Yield the integer when conversion succeeds.
4. Catch `ValueError` and print a short message when conversion fails.
5. Continue with the next value after an invalid entry.

The generator should yield valid scores one at a time. Do not make a separate list of valid scores inside the generator.

## Part 2: Calculate a class summary

Write a function `score_summary(matrix)` that uses the generator and returns a tuple containing:

```
(number_of_valid_scores, total, average)
```

For the example matrix, the result should be:

```
(7, 85, 85 / 7)
```

If the matrix contains no valid scores, return:

```
(0, 0, 0)
```

The function must consume the generator with a loop. Do not convert the generator to a list.

## Part 3: Test missing scores

Test these matrices:

```
empty_scores = []

invalid_scores = [
    ["missing", "absent"],
    ["not submitted"],
]
```

The program should return `(0, 0, 0)` for both cases and should not crash.

## Example Run

Possible output when testing the example matrix:

```
Skipping invalid score: missing
Skipping invalid score: absent
(7, 85, 12.142857142857142)
```

The exact formatting of the average may be different.

## Requirements

- Use a matrix represented as a list of lists.
- Use nested loops to visit every matrix element.
- Define and use a generator containing `yield`.
- Use `try` and `except ValueError` for invalid score strings.
- Continue processing after invalid entries.
- Do not convert the generator to a list.
- Return the count, total, and average of valid scores.
- Handle an empty matrix and a matrix with no valid scores.

## Discussion Questions

1. Why do we need two loops to process the matrix?
2. What does `yield` do in `valid_scores()`?
3. When does the generator start executing?
4. Why does `int("missing")` raise `ValueError`?
5. What is the data flow from the matrix to the class average?
6. Which parts of the program control the order in which operations happen?

## Extension Challenges

1. Reject scores below 0 or above 20 by raising and handling `ValueError`.
2. Write a generator that yields only passing scores, where a pass is 10 or higher.
3. Return the highest and lowest valid scores.
4. Print the row number when an invalid score is skipped.